

CORPORACIÓN UNIVERSITARIA MINUTO DE DIOS – UNIMINUTO

Rectoría UNIMINUTO Virtual

**Modelado digital 3D: sistema de software con integración del
aseguramiento de la calidad**

Caso: sistema de gestión de tickets de soporte para mesa de ayuda (STS)

Kevin Nicolás Buenaventura Ramírez

ID estudiantil: 432270

Estándares y Métricas de Calidad en el Desarrollo de Software
NRC-93980

Docente: Cesar Alfonso Bolado Silva

Bogotá D.C., 2026

Introducción

Este trabajo tiene como objetivo elaborar el modelado digital de un sistema de software desde la perspectiva del aseguramiento de la calidad, aplicando notación UML para especificar de manera estructurada sus requisitos, su comportamiento y su estructura conceptual. El sistema propuesto es STS (Sistema de Tickets de Soporte), una plataforma para la gestión de incidentes reportados a una mesa de ayuda, en la que cada ticket recibe una prioridad, un tiempo de atención (SLA) y un seguimiento hasta su cierre.

Más allá de construir los diagramas, el propósito del modelo es demostrar cómo el aseguramiento de la calidad se integra desde la fase de especificación: mediante la definición precisa de requisitos funcionales y no funcionales, criterios de aceptación verificables, trazabilidad básica entre requisitos y casos de prueba, y puntos de control de calidad distribuidos a lo largo del ciclo de vida del software, de manera que la calidad no dependa únicamente de las pruebas finales, sino que esté prevista desde el diseño.

Desarrollo

Descripción del sistema propuesto

STS (Sistema de Tickets de Soporte) es una aplicación de gestión de incidentes para una mesa de ayuda, estructurada conceptualmente en tres capas: modelo (Ticket, Usuario y sus especializaciones), servicio (TicketService, NotificacionService) y control de acceso (roles Solicitante, Agente de soporte y Administrador de mesa de ayuda). El escenario elegido responde directamente a mi contexto profesional: en mi labor como analista de soporte y desarrollo, la correcta priorización y trazabilidad de los tickets reportados es un factor determinante de la calidad del servicio, por lo que este sistema no es un ejercicio abstracto, sino la especificación de una herramienta con aplicación real en la operación de una mesa de ayuda.

Requisitos funcionales

ID	Descripción
RF-01	El sistema debe permitir a un solicitante registrar un nuevo ticket indicando título, descripción y prioridad.
RF-02	El sistema debe calcular automáticamente el SLA (horas de atención) según la prioridad del ticket (ALTA=4h, MEDIA=24h, BAJA=72h).
RF-03	El sistema debe permitir a un administrador asignar un ticket a un agente de soporte.
RF-04	El sistema debe permitir a un agente cerrar un ticket, registrando la fecha y hora de cierre.
RF-05	El sistema debe permitir a cualquier usuario autenticado consultar el estado actual de un ticket.
RF-06	El sistema debe escalar automáticamente un ticket cuyo SLA se haya vencido sin haber sido cerrado.
RF-07	El sistema debe notificar al solicitante por correo o SMS ante cada cambio de estado del ticket.
RF-08	El sistema debe permitir al administrador generar un reporte de cumplimiento de SLA por periodo.

Requisitos no funcionales

ID	Atributo de calidad	Descripción
RNF-01	Disponibilidad	El sistema debe estar disponible al menos el 99 % del horario laboral (7:00 a. m.–9:00 p. m.).
RNF-02	Seguridad	El acceso a los tickets debe requerir autenticación y control de acceso basado en el rol del usuario.
RNF-03	Rendimiento	El cálculo del SLA y la consulta de estado deben responder en menos de 2 segundos.
RNF-04	Mantenibilidad	La lógica de cálculo de SLA debe estar aislada en un componente de servicio, independiente de la interfaz.
RNF-05	Usabilidad	Un agente de soporte debe poder registrar el cierre de un ticket en un máximo de tres pasos.

Estructura del modelo UML elaborado

El modelo se construyó con dos diagramas UML complementarios. El diagrama de casos de uso (sección Modelado, figura 1) representa el comportamiento del sistema desde la perspectiva de sus tres actores humanos (solicitante, agente de soporte y administrador de mesa de ayuda) y un actor externo (servicio de notificaciones), e incluye relaciones «include» que muestran cómo registrar y asignar un ticket dependen internamente de calcular su SLA, y cómo varias acciones del sistema disparan la notificación al solicitante. El diagrama de clases conceptual (figura 2) especifica la estructura estática: una jerarquía de herencia Usuario → Solicitante / AgenteSoporte / AdministradorMesa, la clase central Ticket asociada a las enumeraciones Prioridad y EstadoTicket, y dos clases de servicio (TicketService, NotificacionService) que encapsulan la lógica de negocio y las dependencias entre componentes.

Justificación de las decisiones de diseño

Se optó por modelar Usuario como una clase abstracta con tres especializaciones, en lugar de un único atributo "rol", porque cada tipo de usuario tiene comportamientos y responsabilidades distintas (el solicitante registra, el agente atiende y cierra, el administrador asigna y audita), lo cual es más coherente con el principio de responsabilidad única y facilita la trazabilidad de quién ejecuta cada acción sobre un ticket. De igual forma, se separó la lógica de cálculo de SLA en una clase de servicio (TicketService) independiente del modelo Ticket, siguiendo el mismo patrón Modelo-Servicio-Controlador validado en el análisis técnico de depuración desarrollado previamente en el curso, en el que precisamente un defecto en esa lógica de cálculo de SLA (comparación de prioridad sensible a mayúsculas) generó fallas de trazabilidad. Aislar esta lógica facilita su prueba unitaria de forma independiente y reduce el riesgo de que un cambio en la interfaz afecte el cálculo del SLA.

Integración de mecanismos de aseguramiento de la calidad

El modelo integra el aseguramiento de la calidad en tres niveles. Primero, cada requisito funcional tiene asociado un criterio de aceptación verificable (tabla siguiente), de manera que la validación no dependa de una interpretación subjetiva. Segundo, se estableció una trazabilidad básica entre cada requisito y al menos un caso de prueba, siguiendo el mismo enfoque de trazabilidad requisito–evidencia utilizado en la matriz de verificación y validación del caso MediCitas trabajado en la semana 2. Tercero, se definieron puntos de control de calidad distribuidos a lo largo del ciclo de vida del software (figura 3), de modo que la calidad se verifique de forma incremental —desde la revisión de requisitos hasta la auditoría de cumplimiento de SLA en producción— y no únicamente al final del desarrollo.

Criterios de aceptación

RF	Criterio de aceptación
RF-01	Un ticket registrado con título, descripción y prioridad válidos queda en estado ABIERTO y es consultable de inmediato.
RF-02	Para prioridad ALTA/MEDIA/BAJA, calcularHorasSLA() retorna exactamente 4, 24 y 72 horas, respectivamente, sin distinción de mayúsculas/minúsculas.
RF-03	Tras asignar un ticket, su estado cambia a ASIGNADO y queda vinculado a un único agente.
RF-04	Un ticket cerrado registra fecha de cierre no nula y no puede reabrirse automáticamente.
RF-05	La consulta de estado responde en menos de 2 segundos con el estado y el SLA vigente del ticket.
RF-06	Un ticket con horas transcurridas mayores a su SLA y no cerrado cambia automáticamente a ESCALADO.
RF-07	Cada transición de estado genera un registro de notificación asociado al ticket.
RF-08	El reporte de cumplimiento de SLA totaliza correctamente los tickets vencidos y no vencidos del periodo consultado.

Trazabilidad básica: requisitos y casos de prueba

RF	Caso de prueba	Descripción del caso de prueba
RF-02	CP-01	calcularHorasSLA_prioridadAlta_deberiaRetornar4()
RF-02	CP-02	calcularHorasSLA_prioridadMedia_deberiaRetornar24()
RF-06	CP-03	estaVencido_ticketAltaVencido_deberiaRetornarTrue()
RF-03	CP-04	asignarTicket_conAgenteValido_deberiaCambiarEstadoAAsignado()
RF-04	CP-05	cerrarTicket_deberiaRegistrarFechaDeCierre()
RF-07	CP-06	notificarCambioEstado_deberiaGenerarUnRegistroPorTransicion()

Planeación

El modelo se desarrolló siguiendo cuatro pasos:

- 1. Análisis del dominio: se definió el escenario (mesa de ayuda) a partir de la experiencia profesional del autor, identificando actores, funcionalidades críticas y reglas de negocio (cálculo de SLA por prioridad).
- 2. Especificación de requisitos: se redactaron los requisitos funcionales y no funcionales siguiendo el modelo procedimental de especificación de requisitos de Cornejo-Aparicio et al. (2020), y se les asoció un criterio de aceptación verificable a cada uno.
- 3. Modelado UML: se construyeron el diagrama de casos de uso y el diagrama de clases conceptual, representando actores, relaciones «include»/«extend», jerarquía de herencia, asociaciones y multiplicidades, siguiendo la notación estándar UML 2.5.
- 4. Integración de aseguramiento de la calidad: se construyeron la tabla de trazabilidad requisito–caso de prueba y el diagrama de puntos de control de calidad en el ciclo de vida, alineados con los principios de revisión y auditoría de software (IEEE 1028) trabajados a lo largo del curso.

Herramientas y técnicas utilizadas: notación UML 2.5 (diagrama de casos de uso y diagrama de clases) elaborada de forma vectorial mediante un script de generación de diagramas (equivalente en resultado a herramientas como Draw.io o Lucidchart, con control preciso de la notación); tablas de especificación de requisitos y trazabilidad elaboradas en procesador de texto; y revisión conceptual cruzada con el modelo procedimental de especificación de requisitos de Cornejo-Aparicio et al. (2020) y los criterios de calidad de proceso de Paredes et al. (2017).

Modelado

A continuación se presentan los tres artefactos gráficos que integran el modelo: el diagrama de casos de uso, el diagrama de clases conceptual y el diagrama de puntos de control de calidad en el ciclo de vida. Cada figura se presenta en una página independiente para facilitar su lectura.

Figura 1. Diagrama de casos de uso

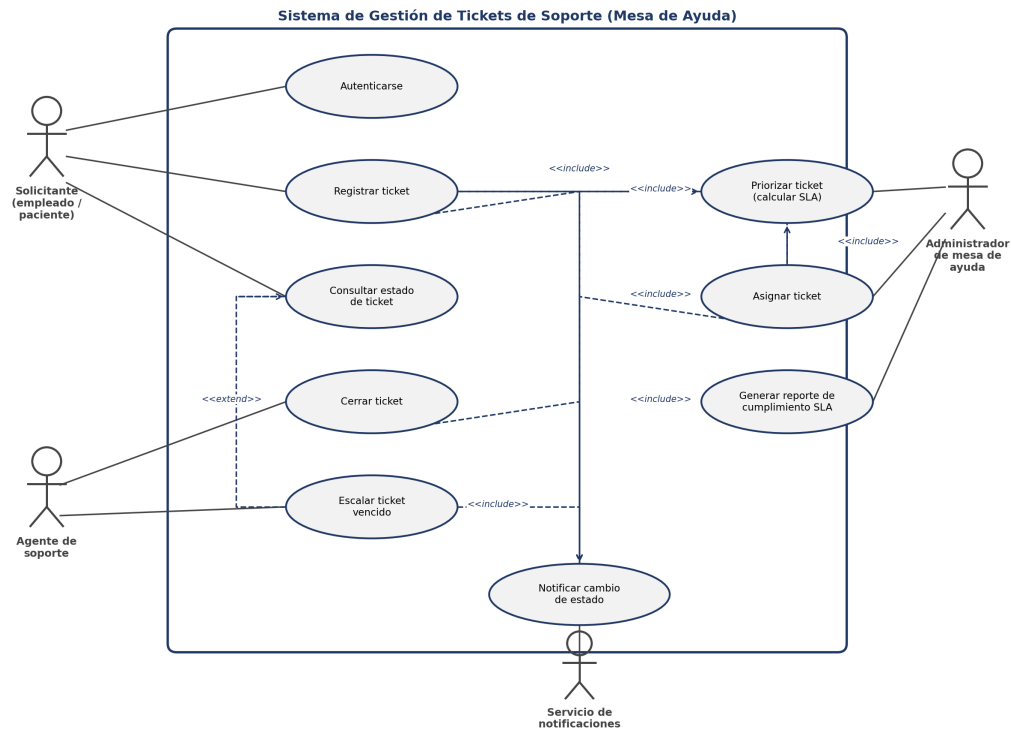


Figura 1. Diagrama de casos de uso del sistema STS (Sistema de Tickets de Soporte).

Figura 2. Diagrama de clases conceptual

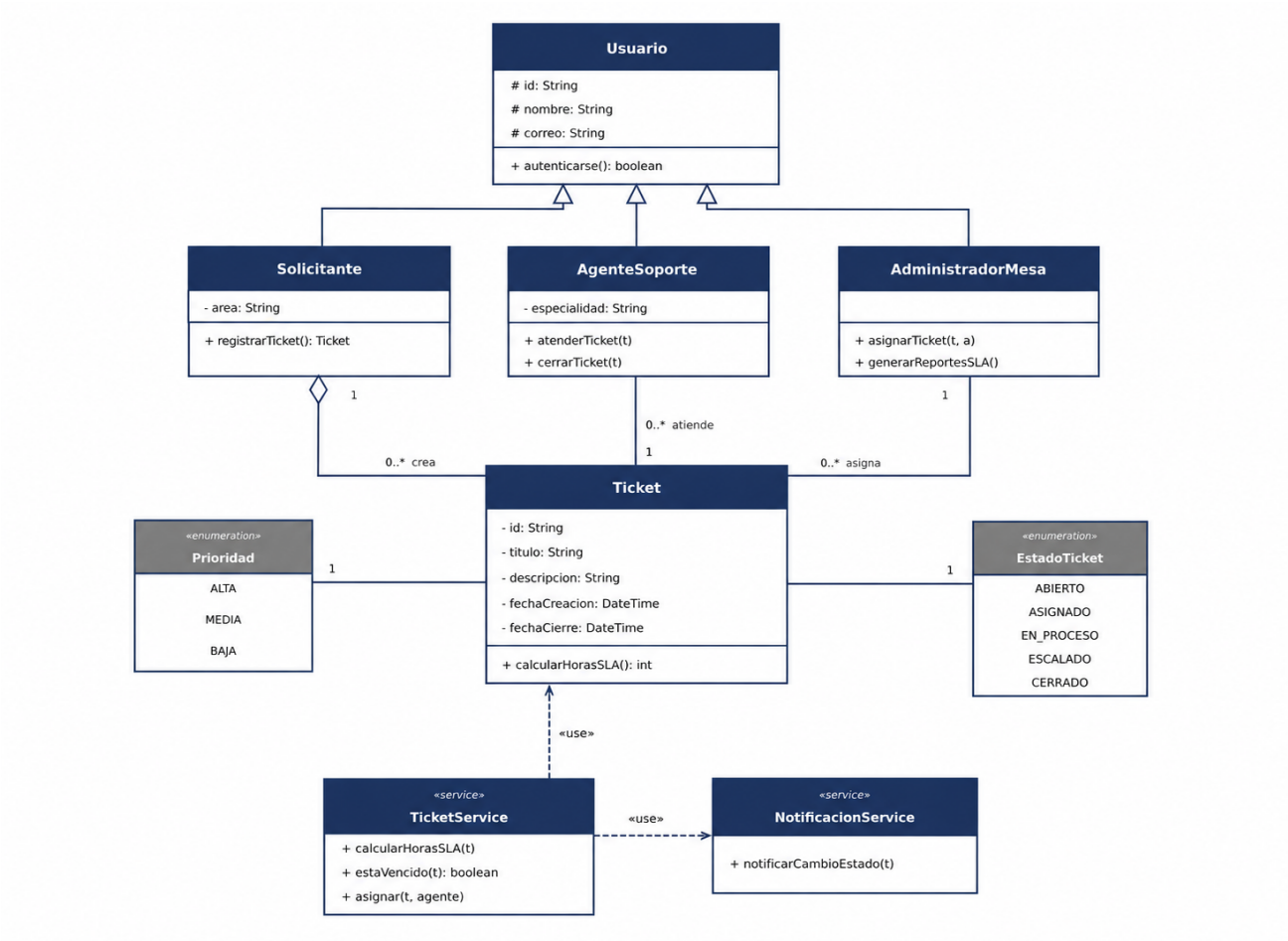


Figura 2. Diagrama de clases conceptual del sistema STS.

Figura 3. Puntos de control de calidad

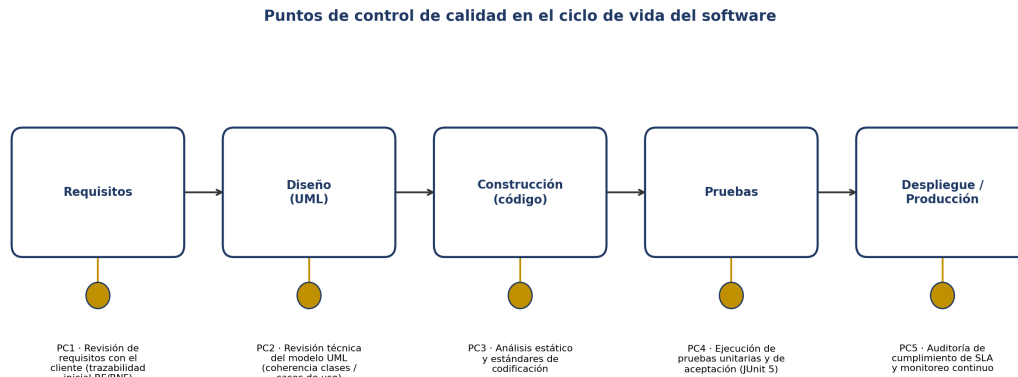


Figura 3. Puntos de control de calidad distribuidos en el ciclo de vida del software.

Conclusiones

Elaborar este modelado permitió comprobar que el aseguramiento de la calidad no es una etapa posterior al desarrollo, sino una dimensión que puede y debe especificarse desde el modelo UML: al asociar cada requisito funcional a un criterio de aceptación y a un caso de prueba desde la fase de diseño, la trazabilidad deja de ser un ejercicio documental aislado y se convierte en parte de la estructura misma del sistema. Asimismo, separar la lógica de negocio (TicketService) del modelo de datos (Ticket) no solo respondió a un principio de diseño, sino que se justificó con evidencia concreta de un ejercicio previo del curso, en el que una falla en esa misma lógica de cálculo de SLA tuvo un impacto real en la calidad del sistema.

Como reflexión final, este ejercicio confirma que modelar correctamente un sistema desde el inicio — con requisitos claros, trazabilidad y puntos de control definidos— reduce significativamente el riesgo de que defectos como los identificados en actividades anteriores del curso lleguen a producción, y constituye una competencia que espero seguir aplicando en mi trabajo diario de soporte y en mi aspiración de liderar la operación de una mesa de ayuda.

Referencias

- Angarita, L. (2021). Aplicación del proceso personal de software en cursos de ingeniería de software. *Revista Ibérica de Sistemas e Tecnologías de Informação*.
- Cornejo-Aparicio, V., Flores-Silva, S., Bedregal-Alpaca, N., & Tupacyupanqui-Jaen, D. (2020). Modelo procedimental para la especificación de requisitos funcionales en la construcción de software. *Revista Ibérica de Sistemas e Tecnologías de Informação*, 571–586.
- Object Management Group. (2017). *OMG Unified Modeling Language (UML)*, versión 2.5.1.

Paredes, L., Benavides, L., & Sánchez, N. (2017). Guía de recomendación para la selección de un modelo de calidad para la mejora de procesos de software (SPI). Revista Ciencias Estratégicas.